

Correct but Not Parallel: What Agent-Generated OpenMP Code Pays to Be Race-Free

Anonymous Author(s)
Affiliation withheld for review
Location withheld
email withheld

Abstract—Agentic AI is increasingly used to write the parallel code at the heart of high-performance computing, raising a trust question: on what basis do we accept what it produces? A natural answer is configuration-robust correctness—agreement with a trusted serial reference under *every* thread count and repeated run—which catches the silent races and order-dependent bugs a single execution hides. We build an agent that writes OpenMP code and can invoke a distribution-aware differential verifier that establishes exactly this property, and evaluate it on the standard ParEval benchmark with four state-of-the-art models. Correctness, it turns out, is no longer the binding constraint: single-shot generation is already robustly correct 82.8% of the time, a repair loop raises this to 87.9%, and replacing the loop’s single-thread check with the full differential verifier leaves it unchanged at 87.9%—these models seldom write the silent-race failure mode the verifier targets, and when they fail they fail already on one thread—a null an independent happens-before race detector (Archer) corroborates on every accepted program. (An open-weights panel confirms the mechanism from the other side: a smaller model ships a genuine shared-bin race the verifier catches and repairs, so the tool’s value returns as capability falls.) The sharper finding is that robust correctness is *necessary but not sufficient*, and gameable by construction: a serial program with no parallelism scores a perfect 87.9%-equivalent. Measuring the second axis directly, we find the 63 programs the verifier accepted as robust span the entire scaling range from $7.9\times$ speedup down to $0.33\times$ —one accepted program runs three times *slower* on eight threads than one. Correctness stopped being the hard part; writing code that is correct *and* actually parallel is where agent-generated HPC code now fails, and an acceptance test that scores only outputs cannot see it. We release the harness, prompts, and complete agent transcripts.

Index Terms—agentic AI, large language models, parallel programming, OpenMP, HPC software engineering, differential testing, correctness, data races

I. INTRODUCTION

Agentic AI—systems that plan, reason, and use tools autonomously—is rapidly being adopted to write the parallel and distributed code at the heart of high-performance and scientific computing: refactoring legacy kernels, translating across programming models, and generating the data-processing stages that connect simulation, analytics, and machine learning [1]. The productivity case is strong. The reliability question is new and sharp: when an agent writes code whose correctness depends on *how it is executed in parallel*, on what basis do we trust what it produced?

Two properties make this hard. First, the correctness of parallel and distributed code is a *global* property of its execution.

Whether a result is right can depend on how many partitions or threads the work is divided into, whether an operation imposes a total order, and how partial results are combined across workers—none of which is visible in the *local* sequence of tokens a code model attends to. The difficulty is documented: state-of-the-art models are far less reliable at parallel code than at its serial equivalent [2], and generated parallel routines routinely harbor races and nondeterminism that a casual run does not reveal. Second, and specific to *agentic* systems: an agent that checks its own work by *executing it once* observes that the program ran and produced plausible output, and concludes it is correct. But distribution- and concurrency-semantics bugs need not fail on any single run. The very signal an agent naturally uses to verify itself can be blind to a class of bugs that only a change of configuration reveals—and so is the check a developer most often performs.

Consider an agent asked to compute a histogram—count how many array elements fall into each of several bins. The agent writes an OpenMP loop that increments a shared bin array in parallel, runs it once, sees a plausible histogram, and submits. The program is wrong: multiple threads update the same bin counter without synchronization, so counts are lost to a data race. But on a single thread the loop runs sequentially and the histogram is exactly right, and even on several threads the lost updates may be few enough to look plausible. Nothing in a single run reliably reveals this—not to a developer who validated once, and not to an agent whose only tool is to execute and inspect the output. The bug is invisible to exactly the check that was performed.

We frame the property such a check should establish as *configuration-robust correctness*: a generated program is robustly correct only if it agrees with a trusted serial reference under *every* execution configuration—each thread count and repeated run—rather than under the single configuration that happens to be tested. The quantity of interest is the *gap* between *single-configuration correctness*, what an ordinary run observes, and *configuration-robust correctness*: programs in this gap pass the check yet are not robust. This gap is the silent-failure surface—and, crucially for an agent, it is *actionable*: the right verification tool can expose it.

A natural hypothesis is that closing this gap is primarily a matter of *verification*: an agent that can differentially execute its code across configurations should detect the programs in the gap and repair them, whereas an agent limited to single-

configuration execution cannot. Existing self-repair loops—Self-Debugging [3], Reflexion [4], and, in HPC, self-correcting parallel-code translation [5]—close the loop but consume exactly the single-configuration signal that is, in principle, blind to these bugs. This motivates two questions. First, the one we set out to answer: *how often does agent-generated parallel code fall into this gap, and does distribution-aware differential verification measurably improve robust correctness?* The answer, on standard tasks with frontier models, is a clarifying *no*—correctness is already largely solved. That answer forces the second, sharper question, which becomes the heart of the paper: *if these programs are robustly correct, are they actually parallel?* A robustness metric defined on outputs is silent here—and maximized by deleting every parallel construct—so we measure the performance axis directly and find that this, not correctness, is where accepted code now fails.

We answer with an open, reproducible study built on PAREVAL [2], the standard benchmark for LLM-generated parallel code. A tool-using agent writes OpenMP code and can invoke a small toolbox: compile and run a draft at one thread count, or *differentially verify* it across a thread-count sweep with repeated runs against PAREVAL’s serial reference. Compilation and execution run on a multi-core cloud machine. We evaluate the agent with state-of-the-art models and, in a controlled *tool-ablation*, remove the differential verifier to isolate its contribution; final correctness is always scored by an independent verifier over the full thread sweep, so the outcome measure does not depend on which tools the agent was given. This paper makes five contributions. *First*, we formalize *configuration-robust correctness* and the *single-configuration-to-robust gap* as the evaluation lens for agent-generated parallel and distributed code, distinct from single-execution functional correctness (Section II). *Second*, we build a distribution-aware differential verifier and expose it as a first-class tool inside a tool-using agent that writes and self-repairs parallel code; we release the harness, tasks, prompts, and complete agent transcripts in line with the SC26 reproducibility initiative (Sections II–III). *Third*, we evaluate the agent with state-of-the-art models on ParEval’s OpenMP tasks, reporting configuration-robust correctness with repeated runs and Wilson confidence intervals (Section IV). *Fourth*, our controlled tool-ablation returns a *negative and clarifying* result: for 4 state-of-the-art models on standard OpenMP tasks, the differential verifier does not improve over single-configuration execution, because these models seldom emit the silent-race failure mode it targets; their errors are semantic and already visible at one thread, and it is the agent’s *repair loop*, not the verifier’s granularity, that drives the improvement over single-shot generation. A positive control with injected races and an open-weights panel—on which a smaller model ships a genuine race the verifier catches and repairs—bound the tool’s value: it rises exactly as capability falls (Sections IV–V). *Fifth*, and most consequential, we show that configuration-robust correctness is *necessary but not sufficient* and gameable by construction—a serial program scores perfectly—and measure the second, performance axis directly: the programs the ver-

ifier accepts span $0.33\times$ to $7.9\times$ speedup on eight threads, one running *slower* in parallel than serially. Once frontier models make race-freedom cheap, “correct but not parallel” is where agent-generated HPC code fails, and an output-only acceptance test cannot see it (Section IV-G).

II. CONFIGURATION-ROBUST CORRECTNESS

We formalize the metrics used throughout the paper. The guiding idea is to separate correctness that a developer (or an agent) can *observe cheaply*—by running a program once at low parallelism—from correctness that actually *holds* once the program executes across the parallel configurations a production run will use.

A. Setup and Notation

Let $\mathcal{M}$ denote the set of models under evaluation and $\mathcal{T}$ a set of parallel-programming tasks. Each task $t \in \mathcal{T}$ is specified by a natural-language prompt with an explicit function contract and is equipped with a *trusted serial reference* R_t : a sequential implementation that defines the correct output on any input. A model m produces a program

$$G(m, t) = \text{EXTRACT}(m(\text{PROMPT}(t))), \quad (1)$$

where $\text{EXTRACT}(\cdot)$ recovers the code from the response. The generated program and the reference are compiled into the same driver, which supplies randomized inputs and adjudicates the program’s output against R_t .

A parallel program admits a family of *execution configurations* $\mathcal{K}$. A configuration fixes a degree of parallelism—here an OpenMP thread count $\tau \in \{1, 2, 4, 8\}$ —and a repetition index that re-runs the same program to expose nondeterminism. Executing program G under configuration $k \in \mathcal{K}$ yields either a validated output or the failure symbol $\perp$ (a crash or a compilation failure), and the driver reports a per-configuration correctness indicator

$$\text{corr}(G, k) = \begin{cases} 1 & G \text{ runs under } k \text{ and its output matches } R_t \\ & \text{on every validation input,} \\ 0 & \text{otherwise.} \end{cases} \quad (2)$$

Correctness is checked against the serial reference on multiple randomized inputs, so a program must be right on all of them, not merely on a fixed example.

B. Single-Configuration vs. Robust Correctness

Let $k_0 \in \mathcal{K}$ denote the *developer-default* configuration: a single thread ($\tau=1$), the most forgiving setting and the one a quick local check most naturally uses. Because a single-threaded run executes the parallel constructs sequentially, it hides precisely the hazards—races, non-associative combines, order dependence—that only manifest when the work is actually divided. We define two correctness notions for a generated program G :

$$\text{single-configuration: } \mathcal{S}(G) = \text{corr}(G, k_0), \quad (3)$$

$$\text{configuration-robust: } \mathcal{R}(G) = \prod_{k \in \mathcal{K}} \text{corr}(G, k). \quad (4)$$

$\mathcal{S}$ is what an ordinary single-run check observes; $\mathcal{R}$ requires agreement with the reference under *every* thread count and every repetition. We also define *configuration-invariance* $\mathcal{I}(G) = 1$ iff G produces identical outputs for all k (regardless of whether they are correct)—a program can be invariant yet uniformly wrong.

C. Correctness Is Necessary, Not Sufficient: The Second Axis

$\mathcal{R}$ has a deliberate blind spot that is central to this paper: it is a predicate on *outputs*, so it is *maximized by removing parallelism altogether*. A program that deletes every `#pragma`, or wraps the loop body in one `critical`, agrees with the serial reference at every thread count and is therefore configuration-robustly correct—while running no faster, or slower, than serial code. A correctness metric for parallel code that a serial program aces is not, by itself, measuring parallel code. We therefore pair $\mathcal{R}$ with a *performance* axis. For a program that compiles and validates, let $T(G, p)$ be its wall-time on p threads; its *self-speedup* is $\mathcal{P}_p(G) = T(G, 1)/T(G, p)$ and its parallel efficiency $\mathcal{P}_p(G)/p$. An acceptance test for agent-generated parallel code needs *both* axes: $\mathcal{R}(G)=1$ (robustly correct) and $\mathcal{P}_p(G)$ meaningfully above 1 (actually parallel). Section IV shows the second axis, not the first, is where accepted code fails.

D. $\mathcal{R}$ Is an Estimator with Bounded Power

$\mathcal{R}$ in Eq. 4 is a finite product over a sampled $\mathcal{K}$, not a decision over the whole configuration space, so it is an *estimator* of robustness with quantifiable power. If a program fails a given configuration nondeterministically with per-run probability p , then N independent validations detect it with probability $1 - (1 - p)^N$; conversely, observing 0 failures over N runs bounds the latent failure probability above by $\approx 3/N$ at 95% confidence (the rule of three). We use this throughout: we report our null not as “zero races” but as an upper bound on the silent-failure risk, and we characterize the estimator’s power against injected hazards as a function of input size, thread count, and repetitions (Section IV).

E. The Parallel-Correctness Gap

Aggregating over a population of generated programs, the *robust-correctness rate* is $\overline{\mathcal{R}} = \mathbb{E}[\mathcal{R}(G)]$ and the *single-configuration rate* is $\overline{\mathcal{S}} = \mathbb{E}[\mathcal{S}(G)]$. Their difference isolates programs that pass the developer-default check yet are not robustly correct:

$$\Delta = \Pr[\mathcal{S}(G)=1 \wedge \mathcal{R}(G)=0], \quad (5)$$

the *parallel-correctness gap*, with associated conditional $\Pr[\mathcal{R}=0 \mid \mathcal{S}=1]$, the *silent-failure risk*: of programs that look correct at low parallelism, the fraction actually wrong at some higher thread count. This gap is the surface of bugs that neither a developer’s single run nor an agent’s single-execution check can see.

TABLE I
PARALLEL HAZARD CLASSES PROBED BY THE BENCHMARK AND THEIR SIGNATURE UNDER DIFFERENTIAL (THREAD-SWEPT, REPEATED) EXECUTION. INV. = THE WRONG RESULT IS INVARIANT ACROSS THREAD COUNTS; *varies* = THE RESULT CHANGES WITH THE THREAD COUNT OR ACROSS REPETITIONS.

Class	Typical cause	signature
H1 data race	shared accumulator written without reduction/atomic	<i>varies</i>
H2 non-associative combine	reduction whose operator is not associative/commutative as written	<i>varies</i>
H3 order-dependent selection	“first”/representative element chosen without a total order	<i>varies</i>
H4 sequential dependence	scan/recurrence parallelized despite a loop-carried dependence	invariant/ <i>varies</i>

F. Verdicts and Hazard Classes

For reporting we assign each program a mutually exclusive verdict: *robust* ($\mathcal{R}=1$); *flaky* ($\exists k : \text{corr}(G, k)=1$ but $\mathcal{R}=0$, i.e. correct at some thread counts or repetitions and not others—the signature of a data race); *wrong* (compiles and runs but never matches the reference); and *crash* (fails to compile or run under every configuration). Orthogonally, we tag each program by the *parallel hazard* it may exhibit, summarized in Table I. Each hazard has a characteristic differential signature—whether the failure varies across thread counts or is invariant—that the harness detects automatically.

G. An Agent and Its Verification Tool

The metrics above measure single-shot generation. To ask whether an *agent* can repair its own parallel code, we wrap generation in a standard propose–observe–repair loop and vary only the *tool* that produces the observation. Given a task, the agent emits a program; a tool returns an observation; the observation is appended to the context; the agent revises. The loop stops when the tool reports success or a repair budget B is exhausted. We define three tool levels that isolate the value of distribution-aware verification:

- **L0 (single-shot).** No tool; the first program is accepted. This is the baseline, $\mathcal{S}/\mathcal{R}$ of Section II.
- **L1 (execute-once).** The program is compiled and run under the developer-default configuration k_0 ($\tau=1$); a crash or compile trace is returned, but the program is validated at only one thread count, so any program that runs and matches at $\tau=1$ is accepted. This is what an ordinary execute-and-fix agent observes.
- **L2 (differential verify).** The verifier of Section II compiles the program and executes it across the thread sweep with repetitions, checking each run against the serial reference R_t and returning PASS or a specific diagnostic (a failing thread count, a race-indicating variation across repetitions, or a crash).

Only the observation differs across levels; the model, prompt, and budget are held fixed, so any difference in final correctness is attributable to the *tool*. We report the robust-correctness rate $\bar{R}$ of each level’s *final* program and the average number of repair rounds it consumes.

III. BENCHMARK AND EXPERIMENTAL SETUP

A. Tasks

We build directly on PAREVAL [2], the standard benchmark for LLM-generated parallel code, reusing its function-completion prompts, its randomized-input drivers, and—crucially—its *serial reference* implementations as our trusted oracle R_t . We draw the OpenMP tasks whose correct result is sensitive to how the work is parallelized: histograms (shared-bin updates), reductions (associativity and commutativity of the combine), scans (loop-carried dependence), and searches (order-dependent selection). These instantiate the hazard classes of Table I. Each prompt states the required computation and signature but never dictates the parallelization strategy, so the model must choose a race-free, order-correct implementation on its own.

B. Oracle and Data

Correctness is adjudicated by PAREVAL’s own driver: for each generated completion the driver compiles the model’s function together with the serial reference, generates randomized inputs, runs both, and reports whether their outputs agree. Using the benchmark’s serial reference as the oracle means the ground truth is independent of the parallel implementation under test, and the randomized multi-input validation prevents a program from passing on a single lucky example. Because the oracle is a sequential program, it is itself immune to the parallel hazards we study.

C. Differential Verifier and Configurations

The novel component of our harness is a *distribution-aware differential verifier*. Rather than validating a completion once, it compiles the program with OpenMP and executes it across a thread-count sweep $\tau \in \{1, 2, 4, 8\}$ with repeated runs at each setting, checking every run against the serial reference. A program is robustly correct only if it validates at every thread count and every repetition (Eq. 4); a result that changes with the thread count, or that varies across repetitions at a fixed thread count, is the signature of a data race or an order-dependent computation and is reported as *not robust* with a diagnostic. Repeated execution is what lets the verifier catch nondeterministic races that a single validation—PAREVAL’s default, and an ordinary agent’s execute-and-fix signal—can miss. Compilation and execution run on a dedicated multi-core cloud VM; the verifier, prompts, and per-run transcripts are released for reproduction in line with the SC26 reproducibility initiative.

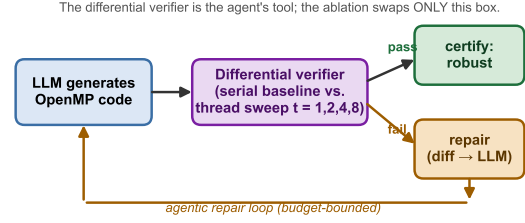


Fig. 1. The agentic pipeline. The model proposes a completion; the tool returns an observation (nothing at L0, a single-thread pass/fail at L1, or the full thread-sweep verdict at L2); on failure the agent repairs until the tool reports success or the budget is spent. Every run’s *final* program is scored by the full differential verifier, independent of the tool the agent could use.

D. Model Panel

We evaluate 4 state-of-the-art instruction-tuned models from a single provider, spanning a current frontier general-purpose model, its preceding generation, a code-specialized model, and a higher-reasoning “pro” variant—all accessed through Azure AI Foundry under Entra ID authentication, with no API keys stored. Holding the benchmark, harness, and prompts fixed, this within-provider panel spans capability and specialization tiers while controlling for provider-specific idiom: a tool effect—or its absence—that holds across all 4, including a model tuned specifically for code, is unlikely to be an artifact of a single weak or idiosyncratic model. Our harness is model-agnostic and reaches any deployment through one uniform API. We exploit this to add an *open-weights* panel—3 models served through an OpenAI-compatible gateway on a research cluster, spanning a 120B general model and a 27B model—which we run on the race-prone task subset to probe the capability-dependence of the verifier’s value (Section IV-F); a broader cross-vendor sweep remains future work (Section V). Generation in the baseline condition is single-shot with no tool use: the model receives the prompt and returns a completion, which the harness compiles and verifies as-is. Every generation—system prompt, user prompt, raw response, extracted code, and verifier feedback—is logged as a reusable artifact.

E. Agentic Tool-Ablation Setup

For the tool-ablation (Section II-G) we run the same propose–observe–repair agent at levels L0 (single-shot), L1 (execute-once), and L2 (differential verify), holding the model, prompt, and repair budget B fixed and varying only the verification tool (Fig. 1). The agent completes the task, the tool returns an observation, and—for L1 and L2—the agent repairs on failure until the tool reports success or the budget is exhausted. Regardless of condition, the *final* program of every run is scored by the full differential verifier over the complete thread sweep, an outcome measure independent of which tool the agent was permitted to use. Every repair round—the verifier feedback and the revised program—is logged, so the agent’s tool-use and repair trajectory is fully auditable.

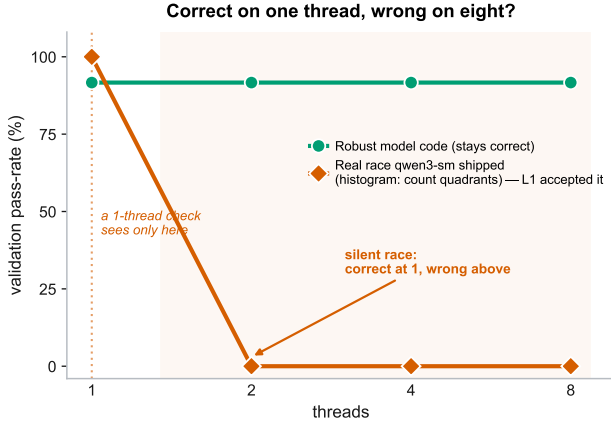


Fig. 2. The study in one plot. Validation pass-rate versus thread count. Robust model code (green) stays correct at every thread count; the red curve is a *real* silent race an open-weights model shipped—a shared-bin histogram that is correct on one thread and wrong on two, four, and eight (Section IV-F), which the execute-once tool accepted. A one-thread check sees only $\tau=1$ (shaded region is invisible to it).

IV. EVALUATION

We evaluate over 174 agentic runs—16 PAREVAL OpenMP tasks spanning 7 problem families, 4 models, and three tool levels—and report configuration-robust correctness (Eq. 4) with Wilson 95% confidence intervals throughout. The evaluation is built around a single question: does a verifier that *observes* many thread counts make an agent’s parallel code more robust than one that observes a single configuration? Our answer is a precise, instrumented *null*—the differential verifier changed no outcome—paired with a positive control that proves the instrument is not inert. We report the null prominently because we designed the study to detect its opposite, and it did not appear.

A. The Measurement Is a Thread Sweep, Not a Single Run

Figure 2 contains the study in one plot. For every generated program we record the validation pass-rate at each thread count $\tau \in \{1, 2, 4, 8\}$ against the serial oracle. Robust model code (green) is *flat*: a program that is correct at one thread stays correct at eight. The red curve is the failure this paper is about, and it is not hypothetical—it is a program an open-weights model actually shipped (Section IV-F): a shared-bin histogram, correct at one thread and collapsing above it. The title of this paper is the gap between those two curves. A single-configuration check—PAREVAL’s default and an ordinary agent’s execute-and-fix signal—samples only the leftmost point, where the two curves coincide; the differential verifier samples the whole line. For the frontier models the two curves stayed together, because those models did not write programs that live in the gap; the red curve shows what happens when a model does.

B. The Parallel-Correctness Gap—Where It Was Not

The decisive quantity for a differential verifier is how many generated programs are of the *silent-race* kind: correct at $\tau=1$

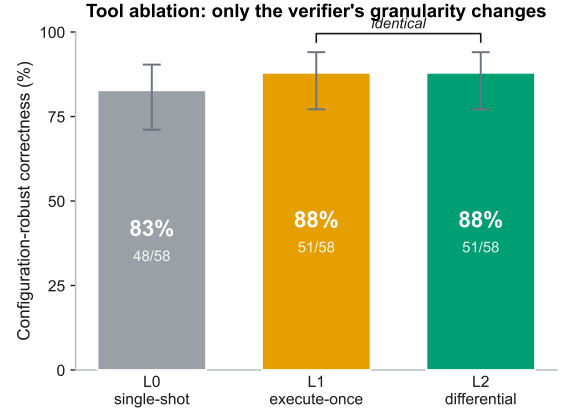


Fig. 3. Tool ablation. Configuration-robust correctness (correct at every thread count and repeat) with Wilson 95% CIs; only the verification tool differs across levels. The repair loop lifts L1 and L2 above single-shot L0, but differential verification (L2) is indistinguishable from execute-once (L1).

yet wrong at some higher thread count. Across all 16×4 task-model cells we observed 0 such programs. When the models were wrong they were wrong already on a single thread—never only at higher thread counts: of 7 failing programs under the strongest tool level, 6 disagreed with the reference already at $\tau=1$ and 1 failed to compile; 0 were races. As Section IV-D details, most of the $\tau=1$ disagreements are a specification-ambiguity artifact on the scan family (correct parallel code, indexed against an ambiguous prompt); the genuine residue is a single reduce task. Where a data race was the natural pitfall—shared histogram bins, a graph accumulator—the models preempted it with the correct OpenMP idiom (a `reduction` or an `atomic`), and those tasks were robust at every thread count. The gap we set out to measure was, for these models on these tasks, empty.

C. Tool Ablation: The Repair Loop Helps; The Verifier’s Granularity Does Not

Figure 3 and Table II report the ablation (Section II-G): the same agent at three tool levels differing only in the observation the tool returns. Single-shot generation (L0) is robustly correct 82.8% of the time (95% CI [71.1, 90.4]). Adding a repair loop driven by an *execute-once* tool (L1) raises this to 87.9% (95% CI [77.1, 94.0]). Replacing that tool with the *differential verifier* (L2) yields 87.9% (95% CI [77.1, 94.0])—*identical* to L1 ($\Delta_{L2-L1} = +0.0\%$). The paired view is sharper than the aggregate: of the 58 task-model cells, the number in which execute-once and differential verification reached a *different* final verdict was 0, and the number that only the differential verifier rescued was 0. The multi-thread signal never changed an outcome. The improvement over single-shot is therefore attributable to the *repair loop* that L1 and L2 share, not to the granularity of the verifier: of the 10 cells not robust at L0, execute-once repaired 3 and the differential verifier repaired the same 3. An agent can only fix what its tool lets it see—but here both tools saw the same failures, because those failures were visible at one thread.

TABLE II
AGENTIC TOOL-ABLATION. ROBUST CORRECTNESS WITH WILSON 95%
CIS; L1 AND L2 COINCIDE EXACTLY.

Tool level	robust correct.	95% CI
L0 single-shot	82.8%	[71.1, 90.4]
L1 execute-once	87.9%	[77.1, 94.0]
L2 differential verify	87.9%	[77.1, 94.0]

D. Heterogeneity Across Models and Problem Families

The equality of L1 and L2 is not an averaging artifact. It holds for every model, including the code-specialized one: no model produced a single case in which the thread sweep and the single-thread check disagreed, even though the newer models failed *more* often overall—their losses were semantic and compile errors, not races. The residual failures were concentrated and, in one family, spurious. Under differential verification the 6+1 non-race failures break down as 4 on the scan family and only 3 elsewhere (all one reduce task, a product-of-inverses combine). We inspected the scan failures by hand: every model computed the reverse-indexed *suffix* sum $\text{out}[i] = \sum_{j \geq i} x_j$ —a defensible reading of “reverse prefix sum”—which is the exact element-wise reverse of PAREVAL’s reference (itself an inclusive scan of the reversed input). The models parallelized correctly; they resolved an ambiguous specification against the reference’s example. Excluding this benchmark artifact, differential-verify robust correctness rises from 87.9% to 94.4%, and the only genuine failures are three instances of a single reduce task—so the barrier to correct parallel code here was getting the sequential algorithm right, not parallelizing it safely.

E. Positive Control: What the Verifier Catches When the Bug Is There

That the differential verifier changed no outcome invites the obvious question: does it work at all? To confirm the instrument is not inert we hand-wrote the *racy* completion a naive or weaker generator would produce for 3 tasks—one per hazard class: an unsynchronized shared-bin histogram, a reduction accumulated without a `reduction` clause, and a first-match search with a racy check-then-write on a shared index. Each is correct on one thread. We verified each twice with the study’s own tool. The execute-once check certified all 3 as robustly correct; the differential verifier rejected all 3, each failing every sweep run above one thread while passing at one thread (Table III). The verifier catches exactly the failure mode a single-thread check is blind to. The null above is therefore a statement about what the models *wrote*, not about what the verifier can *see*.

An independent race detector agrees. A skeptic might still ask whether output-differential testing at $\tau \leq 8$ simply lacks the power to expose races that are present. We therefore cross-checked every one of the 63 accepted programs with a happens-before detector—LLVM Archer [6], [7], ThreadSanitizer instrumented with the OpenMP runtime’s OMPT

TABLE III
POSITIVE CONTROL: INJECTED RACES, ONE PER HAZARD CLASS. EACH IS
CORRECT ON ONE THREAD; EXECUTE-ONCE CERTIFIES ALL, THE
DIFFERENTIAL VERIFIER REJECTS ALL.

Injected hazard	execute-once	differential
Histogram, unsynchronized bins	ACCEPT	REJECT
Reduction, no <code>reduction</code> clause	ACCEPT	REJECT
First-match, racy shared index	ACCEPT	REJECT

annotations—which reasons about the actual synchronization, not just the output. Archer flagged 0 data races in the model-written code of the 63 programs the differential verifier certified robust, while correctly reporting the race in the injected controls and in the open-model histogram of Section IV-F (27 stack frames in the generated code). Two independent instruments—one output-differential, one happens-before—agree that these accepted programs are race-free, which makes the null a statement about the code and not an artifact of a weak detector.

F. Beyond the Frontier: A Real Race the Verifier Catches

The null above is a statement about *capable* models. If differential verification earns its place by catching silent races, the sharpest test is a generator that actually writes them. We therefore replayed the 6 most race-prone tasks through an *open-weights* panel served on a research cluster (3 models ranging from a 120B general model down to a 27B model), holding the harness, prompts, and differential scorer fixed, and scored every final program over the full thread sweep (Fig. 4). The capability gradient is exactly the one the null predicts. The larger open models behaved like the frontier: no silent races, $L1 = L2$. The smallest model did *not*. On the shared-bin quadrant histogram it emitted the canonical hazard—a `local_bins` accumulator declared *outside* the parallel region and incremented by every thread, so the counts are correct on one thread and lost to a data race above it. The execute-once tool ran it once, saw a plausible histogram, and **certified it robust**; the differential verifier ran the sweep, saw it pass at one thread and fail at two, four, and eight, returned that diagnostic, and the agent repaired it to a per-thread private accumulator that is robust throughout. Same model, same task, same repair budget: the only difference was whether the tool looked at more than one thread count. This is the first *model-emitted* instance—not an injected control—of the exact failure this paper is about. Across the open panel the differential verifier caught 1 such verified silent race(s) that the single-thread check certified, versus 0 on the frontier panel (Fig. 4b); we attribute only genuine `pass@1/fail@N` cases to the verifier and treat the remaining small $L1$ – $L2$ differences on this six-task subset as repair-sampling noise, not race catches. The mechanism is what matters: the tool’s value reappears precisely as model capability falls.

Differential verification catches the silent races only weaker models emit

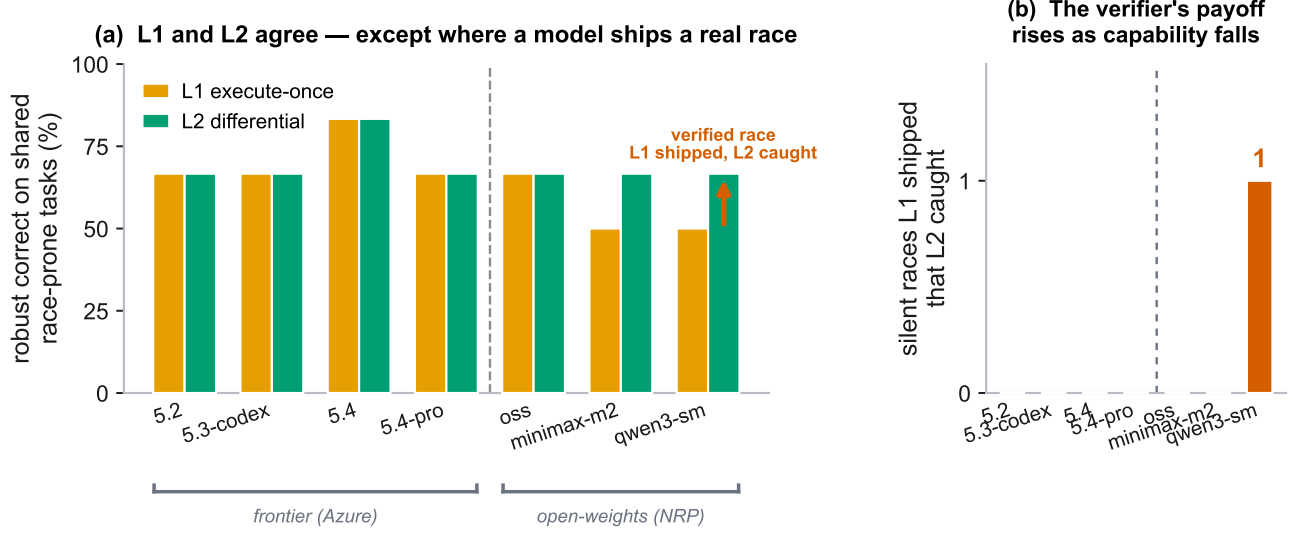


Fig. 4. The verifier’s value is capability-dependent. (a) Robust correctness on the shared race-prone tasks, execute-once (L1) vs. differential (L2), per model. Frontier and larger open models show no meaningful gap; the arrow marks the one *verified* silent race—a shared-bin histogram the smallest open model shipped, which L1 certified and L2 caught and repaired. (b) Verified silent races the single-thread check accepted that the differential verifier rejected: zero for capable models, rising as capability falls. (Small residual L1–L2 differences in (a) with no bar in (b), e.g. minimax-m2, are repair-sampling noise on this six-task subset, not race catches.)

TABLE IV

OPEN-WEIGHTS PANEL ON THE 6 RACE-PRONE TASKS. ROBUST CORRECTNESS UNDER L1 (EXECUTE-ONCE) AND L2 (DIFFERENTIAL); SILENT RACES EACH MODEL SHIPPED (CORRECT AT ONE THREAD, WRONG ABOVE) THAT L1 CERTIFIED; AND HOW MANY L2 CAUGHT. ONLY THE SMALLEST MODEL EMITS A GENUINE RACE—AND L2 CATCHES IT.

Open model	L1	L2	races shipped	caught by L2
gpt-oss-120B	67%	67%	0	0
MiniMax-M2	50%	67%	0	0
Qwen3-27B	50%	67%	1	1

G. Correct but Not Parallel: The Second Axis Is Where Accepted Code Fails

The differential verifier settles the correctness axis; it is silent on the second axis of Section II-C. We measured it directly: for every one of the 63 programs the verifier accepted as robust, we compiled the model’s code against PAREVAL’s timing driver at a size large enough to amortize thread overhead and recorded self-speedup $\mathcal{P}_p$ across the same sweep $p \in \{1, 2, 4, 8\}$ (min wall-time over repeats). The result (Fig. 6) is the paper’s sharpest finding. Robust correctness spans *the entire scaling range*, from $7.9\times$ down to $0.33\times$ —one accepted program runs three times *slower* on eight threads than on one. 24% of accepted programs (15/63) achieve under $2\times$ on eight cores; a serial program, we reiterate, would have been accepted with identical $\mathcal{R}=1$. The decisive juxtaposition (Fig. 6a) is two programs the verifier certified *identically* robust on the same histogram task: one uses a per-thread reduction and scales $6.5\times$; the other guards every bin update with a per-element `atomic` and, throttled by memory-

L1 execute-once — shipped; the 1-thread check passed

```
void countQuadrants(vector<Point> const& pts,
    array<size_t,4>& bins){
    bins = {0,0,0,0};
    array<size_t,4> local_bins; // shared: declared OUTSIDE
    omp parallel
    { #pragma omp parallel
      { local_bins = {0,0,0,0};
        #pragma omp for nowait
        for (size_t i=0;i<pts.size();++i) local_bins[q(pts[i])]
          ]++; // race
        #pragma omp critical
        for(int k=0;k<4;k++) bins[k]+=local_bins[k]; }}
```

verifier: t1=2/2 t2=0/2 t4=0/2 t8=0/2 ⇒ NOT_ROBUST

L2 differential — the sweep failed; the agent repaired it

```
#pragma omp parallel
{ array<size_t,4> local_bins={0,0,0,0}; // private:
  declared INSIDE omp parallel
  #pragma omp for nowait
  for (size_t i=0;i<pts.size();++i) local_bins[q(pts[i])]
    ]++;
  #pragma omp critical
  for(int k=0;k<4;k++) bins[k]+=local_bins[k]; }
```

verifier: t1=2/2 t2=2/2 t4=2/2 t8=2/2 ⇒ ROBUST

Fig. 5. The verified race, from the logs (Qwen3-27B on `count_quadrants`; transcripts released). **Top:** the execute-once agent declares the accumulator *outside* `omp parallel`—so all threads share it—but its one-thread check passes, and it ships. **Bottom:** given the differential tool, the same agent sees the thread sweep fail above one thread and moves the declaration *inside* the region (per-thread private), which is robust. Sweep lines are the tool’s real logged output.

ordering contention, degrades to $0.33\times$. Correctness cannot tell them apart; only the performance axis can.

We are deliberately careful about mechanism. The synchronization idiom a model chose does *not* cleanly predict poor scaling: programs using `critical` mostly used it for a cheap

$O(\text{threads})$ merge of per-thread partials and scaled well (mean $\mathcal{P}_8 = 5.8$ vs. 3.6 for *reduction*), so we do *not* claim models systematically over-synchronize. The defensible claim is narrower and stronger for being exact: an output-only robustness metric is *gameable by construction*, and among accepted programs measured scaling varies enormously—so correctness and parallelism must be reported as two axes, and the second is where the failures now live. This turns the correctness null from an endpoint into a setup: once frontier models make race-freedom cheap, the binding constraint moves to writing code that is not merely correct under parallelism but actually uses it.

H. What the Null Does and Does Not Say

The result is bounded by construction. On 16 standard PAREVAL tasks, 4 strong models emit code whose correctness does not depend on the thread count—the `pass@1/fail@N` mode is absent, corroborated by an independent race detector—so a verifier’s multi-thread granularity cannot help. It does *not* say the verifier is useless: it catches the hazard the moment it is present (the injected controls, the open-model race of Section IV-F, and the cliff in Fig. 2). Correctness verification remains the right first gate; frontier models have simply made its cheapest form sufficient on idiomatic tasks. The consequential gap has moved to the second axis (Section IV-G), which no output-only test can close.

V. DISCUSSION

Our results relocate the binding constraint on agent-generated parallel code *twice*. We expected it to lie in *parallelization*—races and order-dependence that a single run hides. For 4 models on standard OpenMP tasks it did not: the models parallelized safely, and where they failed the error was *algorithmic* and visible on the very first thread. But the deeper relocation is the one the performance axis reveals: once correctness is largely solved, the constraint moves to *parallelism itself*. The programs the verifier accepts span $0.33\times$ to $7.9\times$ speedup (Section IV-G); a robustness metric defined on outputs cannot see the difference, because it is maximized by serial code. Correctness stopped being the hard part, and an acceptance test that scores only outputs is now measuring the wrong axis.

The repair loop, not the verifier’s granularity, was the active ingredient. Holding the model, prompt, and repair budget fixed and varying only the verification tool moves robust correctness from 82.8% (no tool) to 87.9% once a repair loop is present—and then *not at all*: the differential verifier reaches the same 87.9% ($\Delta_{L2-L1} = +0.0\%$), agreeing with the single-thread check on every one of the 58 paired cells. The gain over single-shot generation is the loop’s, not the verifier’s. This does not contradict the design argument that a single-configuration check is *structurally* blind to nondeterministic races—that remains true—but it shows the argument is only *actionable* when models actually emit such races, which here they did not.

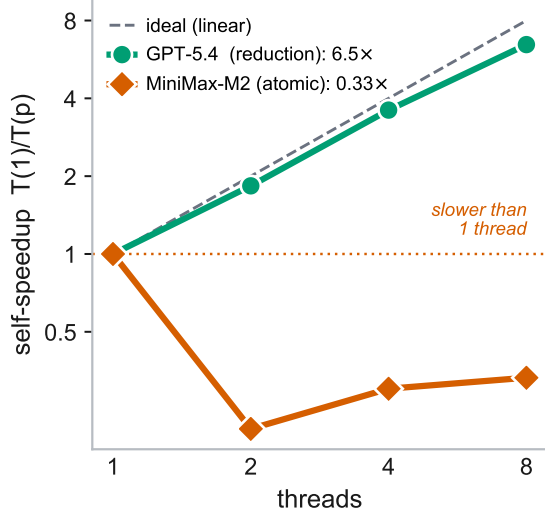
When differential verification pays off. The verifier is cheap—compile once, run a handful of thread counts with a few repetitions against a serial reference—so its cost is a low, fixed premium. What our study measures is the *payout*: zero, when the model $\times$ task distribution produces no silent races. The payout is not zero in general—our injected-race control (Table III) confirms the verifier rejects every racy program the single-thread check accepts. It grows with the race-proneness of that distribution: weaker or non-code-specialized models that reach for manual shared-state updates instead of reduction clauses; tasks whose natural parallelization invites a race rather than a library idiom; and the distributed-memory (MPI) and GPU (CUDA) regimes our harness supports but we did not stress here. The right way to spend the premium is to *measure* it: run the differential gate, watch how often L1 and L2 diverge for your models and tasks, and keep the gate where they do.

Threats to validity. *Construct.* We study OpenMP shared-memory parallelism over ParEval’s tasks and serial references; distributed-memory (MPI) and GPU (CUDA) hazards, though supported by our harness, are left for future work, and the hazard taxonomy is not exhaustive. Our primary race evidence is *output-differential*, but we do not rely on it alone: we cross-checked every accepted program with a happens-before detector (Archer/ThreadSanitizer with OMPT annotations [6], [8]), which agreed—0 races in the 63 certified programs—while correctly flagging our injected and open-model races, so the null is corroborated by an independent instrument rather than an artifact of detector power. We still report it conservatively as a bound: with 0 observed, the silent-failure risk is under $\approx 5\%$ at 95% confidence (rule of three). The performance finding does not share this fragility—wall-time is directly measured. *Internal.* Each task has a single serial reference; because it is sequential, it is immune to the parallel hazards under test, but it fixes the task’s intended semantics. The developer-default configuration ($\tau=1$) is a modeling choice for $\mathcal{S}$, chosen as the most failure-masking single run so that Δ upper-bounds what a lucky local check could hide. *External.* Our 4 models are drawn from a single provider (general, preceding-generation, code-specialized, and higher-reasoning tiers) over 16 tasks; holding capability high sharpens the ablation but makes the near-absence of races unsurprising, and neither the null nor the point estimates need transfer to other vendors, weaker models, larger task sets, or prompt/temperature regimes. That the null held even for a code-specialized model tempers, but does not remove, this threat. Because our harness reaches any model through one uniform API, cross-vendor and weaker-model replication is the most immediate extension. We report Wilson intervals and read point estimates alongside them rather than as precise rankings; with 4×16 cells the intervals are wide.

Recommendations. For teams adopting agentic assistants for HPC software, a repair loop is clearly worth having: it supplied the only improvement we measured. A differential-verification gate—validate across thread counts and repeated runs against a trusted reference, rather than trusting a single successful execution—is cheap insurance we still recommend, but with

Configuration-robust correctness is necessary but not sufficient: a serial program scores 100%

(a) Same task, same “robust” verdict—opposite scaling



(b) Robust correctness spans $0.33\times$ – $7.9\times$ scaling

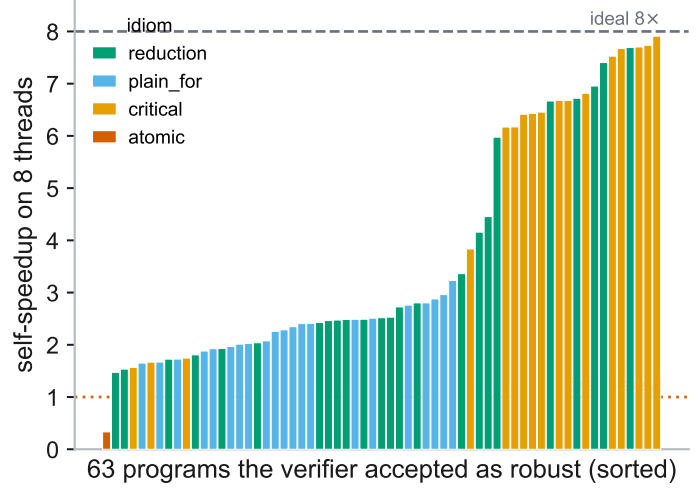


Fig. 6. Correctness is necessary but not sufficient. (a) Two programs the differential verifier accepted as *equally* robust on the same histogram task: a reduction scales $6.5\times$; a per-element atomic runs $0.33\times$ —slower on eight threads than one. (b) Self-speedup on eight threads for all 63 accepted programs, sorted, coloured by synchronization idiom: robust correctness spans $0.33\times$ – $7.9\times$, and a serial program would sit at $1\times$ with the same $\mathcal{R}=1$. The idiom does not cleanly predict scaling (critical is often a cheap merge); the point is that the correctness axis alone cannot distinguish any of these.

eyes open: for capable models on idiomatic tasks its marginal value over a single-thread check may be nil, and the residual failures are algorithmic errors that call for better generation or stronger repair, not finer verification. Treat the gate as instrumentation as much as protection: it is also how you *learn* whether your models write racy code.

VI. RELATED WORK

LLMs for parallel and HPC code. A growing line of work studies whether language models can write parallel code. PAREVAL [2] is the standard benchmark, evaluating $\text{pass}@k$ correctness and speedup for model-generated OpenMP, MPI, CUDA, and Kokkos, and documents a large gap between models’ serial and parallel competence; HPC-CODER [9] fine-tunes models on HPC code, and empirical studies of model-generated OpenMP find frequent semantic errors—missing reductions, wrong data scoping, absent synchronization [10]. These efforts adjudicate correctness on a *single* execution at a fixed degree of parallelism. We build directly on PAREVAL’s tasks and serial references, but change the object of study: rather than scoring one run, we ask whether generated code is correct across the *configuration* axis—thread count and repetition—and whether an agent can be made to repair the failures that this axis exposes. Distributed data-processing pipelines of this kind are the pre- and post-processing stages of AI-coupled HPC workflows [11], where a silent correctness bug propagates into the science.

Functional-correctness evaluation. Code-generation evaluation has centered on functional correctness of small,

self-contained problems—HumanEval [12], its rigorous re-evaluation [13], and MBPP-style suites—and, at repository scale, SWE-bench [14]. All adjudicate a single execution against a fixed test suite. EvalPlus [13] shows that code passing a visible suite often fails under additional tests; our work is the parallel analog along an execution axis developers do not test: the same program, correct at one thread count, fails at another.

Dynamic race detection. A mature line of HPC tools detects data races dynamically: THREADSANITIZER [8] instruments memory accesses at scale, and ARCHER [6], ROMP [15], and SWORD [7] specialize happens-before race detection for OPENMP. These are far more powerful race *detectors* than our output-differential sweep, which only observes end-to-end disagreement. We use them as the reference point for what our estimator can and cannot see (Section II-C): our contribution is not a better race detector but the empirical finding that, for frontier-model output on idiomatic tasks, the race they hunt is largely absent, and the binding constraint has moved to whether the accepted code is parallel at all. Our robustness estimator is also the parallel-code instance of the general *flaky-test* problem—tests whose verdict depends on nondeterministic execution [16]—repetition being our defence against it.

Differential and metamorphic testing. Executing a program under multiple configurations and flagging disagreements is the essence of differential testing [17]; checking properties preserved under transformations is metamorphic testing [18]. In data systems this lineage is rich: query synthesis and reference-

engine construction have found many engine bugs [19], [20], and SPARKFUZZ [21] differentially tests a query engine against a single-node reference. Our verifier applies the differential lens along the *parallelism* axis, and—unlike these offline fuzzers—packages it as a tool an *agent* invokes inside its generation loop: the object under test is the model’s program, not the engine.

Self-repairing and agentic code generation. A large body of work lets models improve their output through execution feedback: Self-Debugging feeds runtime traces back to the model [3], Reflexion adds verbal self-critique [4], and Self-Refine iterates on self-generated feedback [22]; in HPC, LASSI builds a compile-and-run self-correction loop for parallel-code translation [5]. These loops are only as good as the signal they consume. Our tool-ablation isolates exactly this signal quality: in principle an execute-and-fix loop with a single-configuration signal cannot see the nondeterministic parallel bugs that a *differential* signal would catch, so we hold the loop fixed and vary only the verifier—and report, as an honest empirical finding, when that distinction does and does not matter. To our knowledge no prior work packages a distribution-aware differential verifier as an agent tool for parallel-code correctness, nor measures the parallel-correctness gap and its agentic repair on a standard HPC benchmark.

VII. CONCLUSION

Agentic AI will accelerate HPC software engineering only to the extent that we can trust what it writes once that code runs in parallel. We defined *configuration-robust correctness*—agreement with a trusted serial reference under every thread count and repeated run—and built an agent that writes OpenMP code and can invoke a distribution-aware differential verifier that establishes it. On the standard ParEval benchmark with four state-of-the-art models, correctness turned out not to be the binding constraint: single-shot generation is already robustly correct 82.8% of the time, a repair loop raises this to 87.9%, and the differential verifier leaves it unchanged at 87.9% because these models seldom write the silent-race bug it targets—though an open-weights panel shows a smaller model shipping a genuine race the verifier catches and repairs, so its value returns as capability falls. The consequential finding is that robust correctness is *necessary but not sufficient*, and gameable by construction: a serial program scores perfectly, and among the 63 programs the verifier accepted, measured self-speedup spans $0.33\times$ to $7.9\times$ —one running slower in parallel than serially. Once frontier models make race-freedom cheap on idiomatic tasks, “correct but not parallel” is where agent-generated HPC code fails, and an output-only acceptance test cannot see it. The lesson for tool-builders is to measure both axes—correctness *and* scaling—and to treat cross-checking against a dedicated race detector as the way to trust the first. We release the metric, the harness, prompts, and per-generation and per-repair transcripts so that parallelism-aware evaluation of agent-generated HPC code can be studied openly.

ARTIFACT AVAILABILITY

The harness, all task prompts, the model-panel configuration, the differential verifier, and the complete per-generation and per-repair transcripts are released to support reproduction and reuse, in line with the SC26 reproducibility initiative.

REFERENCES

- [1] D. Nichols, A. Marathe, H. Menon, T. Gamblin, and A. Bhatele, “Can large language models write parallel code?” in *HPDC*, 2024.
- [2] D. Nichols *et al.*, “Performance-aligned LLMs for generating fast code,” in *arXiv*, 2024.
- [3] X. Chen, M. Lin, N. Schärli, and D. Zhou, “Teaching large language models to self-debug,” in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2304.05128.
- [4] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2303.11366.
- [5] M. T. Dearing, Y. Tao, X. Wu, Z. Lan, and V. Taylor, “LASSI: An LLM-based automated self-correcting pipeline for translating parallel scientific code,” in *IEEE International Conference on Cluster Computing Workshops (CLUSTER Workshops)*, 2024, arXiv:2407.01638.
- [6] S. Atzeni, G. Gopalakrishnan, Z. Rakamaric, D. H. Ahn, I. Laguna, M. Schulz, G. L. Lee, J. Protze, and M. S. Müller, “ARCHER: effectively spotting data races in large OpenMP applications,” in *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2016.
- [7] S. Atzeni, G. Gopalakrishnan, Z. Rakamaric, I. Laguna, G. L. Lee, and D. H. Ahn, “ARCHER and Sword: dynamic data race detection for OpenMP,” in *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2018.
- [8] K. Serebryany and T. Iskhodzhanov, “ThreadSanitizer: data race detection in practice,” in *Workshop on Binary Instrumentation and Applications (WBIA)*, 2009.
- [9] D. Nichols, A. Marathe, H. Menon, T. Gamblin, and A. Bhatele, “HPC-Coder: Modeling parallel programs using large language models,” in *ISC High Performance*, 2024, arXiv:2306.17281.
- [10] Anonymous, “Can large language models understand parallel programming? an empirical study of OpenMP,” *arXiv preprint arXiv:2311.02359*, 2023.
- [11] W. Brewer, A. Gainaru, M. Emani *et al.*, “AI-coupled HPC workflow applications, middleware and performance,” 2024.
- [12] M. Chen, J. Tworek *et al.*, “Evaluating large language models trained on code,” *arXiv:2107.03374*, 2021.
- [13] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, “Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation,” *NeurIPS*, 2023.
- [14] C. E. Jimenez, J. Yang *et al.*, “SWE-bench: Can language models resolve real-world GitHub issues?” in *ICLR*, 2024.
- [15] Y. Gu and J. Mellor-Crummey, “ROMP: an effective and scalable data race detector for OpenMP programs,” in *ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2018.
- [16] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, “An empirical analysis of flaky tests,” *ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, 2014.
- [17] W. M. McKeeman, “Differential testing for software,” *Digital Technical Journal*, 1998.
- [18] T. Y. Chen, F.-C. Kuo, H. Liu *et al.*, “Metamorphic testing: A review of challenges and opportunities,” *ACM Computing Surveys*, 2018.
- [19] M. Rigger and Z. Su, “Testing database engines via pivoted query synthesis,” in *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020.
- [20] —, “Detecting optimization bugs in database engines via non-optimizing reference engine construction,” in *ESEC/FSE*, 2020.
- [21] B. Ghit, N. Poggi, J. Rosen, R. Xin, and P. Boncz, “SparkFuzz: Searching correctness regressions in modern query engines,” in *Workshop on Testing Database Systems (DBTest)*, 2020.
- [22] A. Madaan, N. Tandon, P. Gupta *et al.*, “Self-refine: Iterative refinement with self-feedback,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2303.17651.